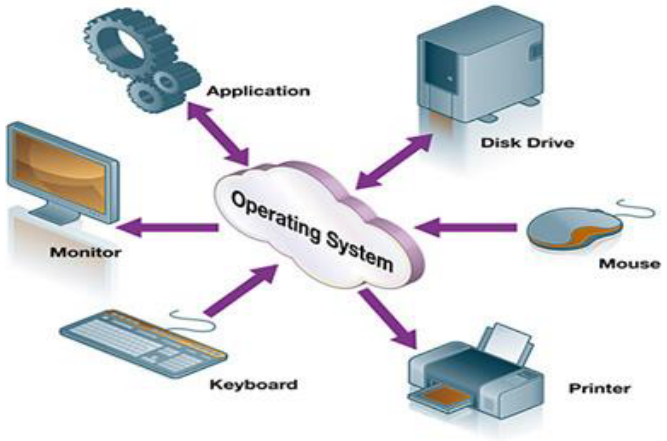
Lecture 1:

Introduction to Operating Systems

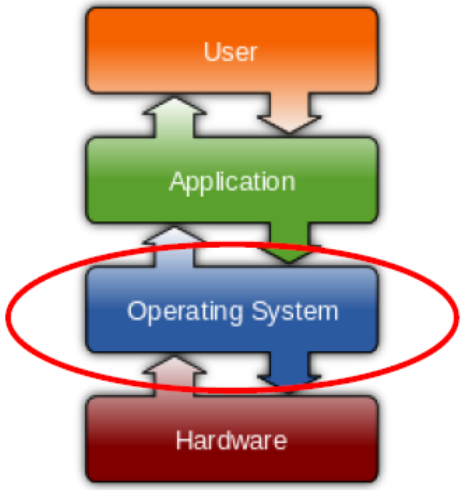


Instructor:

Umesh Hengaju

What is an operating system?

- A type of system control program that manages the resources of the computer system and controls execution of user program.



- It is a program that acts as an intermediary between a user of a computer and the computer hardware.

What is an operating system?

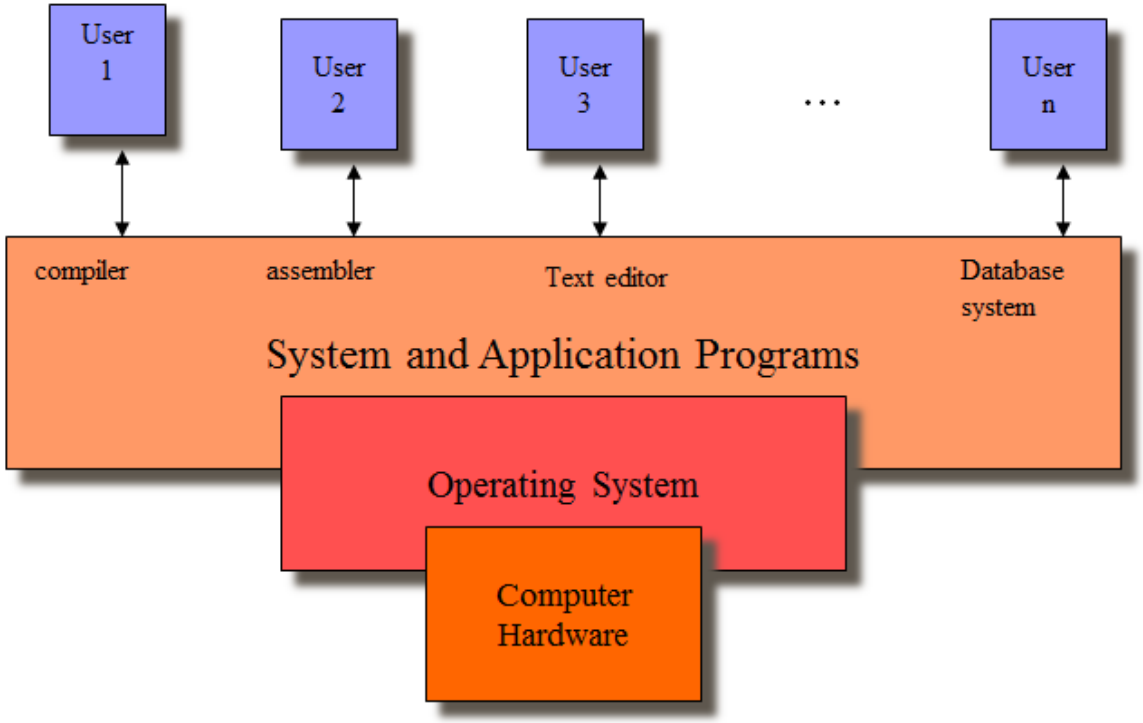
- It is a software layer between application programs and computer hardware.

- It provides a basis and execution environment for application programs.
  - Uniform abstract representation of resources that can be requested and accessed by applications
    i.e. Processor, memory, I/O (disk, network)
  - Exploits the hardware resources of one or more processors
  - Manages secondary memory and I/O devices

Design goals of Operating system:

- Make a computing system convenient to use
- Use the computer hardware and resources in an efficient manner
- Execute user programs and make solving user problems easier
- Create user-friendly computing environment
- Make a computing system secure

Abstract View of Computer System



View of Operating Systems

- ***User Point of View:***
  - Service Provider
- ***System Point of View***
  - Resource Allocator:
    - Exploit the hardware resources of one or more processors and allocate it to user programs
  - Program Controller:
    - Control the execution of programs and operations of I/O devices
  - Protection and Security Provider:
    - Protects multiple programs running from each other

Functions of Operating Systems:

1. ***Process Management:***

- A process is a program in execution. It is a unit of work within the system. Program is a passive entity, and process is an active entity.

- The operating system is responsible for the following activities in connection with process management:
  - Creating and deleting both user and system processes
  - Suspending and resuming processes
  - Providing mechanisms for process synchronization
  - Providing mechanisms for process communication
  - Providing mechanisms for deadlock handling

Functions of Operating Systems:

1. ***Memory Management:***

- All data in memory before and after processing
- All instructions in memory in order to execute
- Memory management determines what is in memory
- Optimizing CPU utilization and computer response to users
- Memory management activities
- Keeping track of which parts of memory are currently being used and by whom
- Deciding which processes (or parts thereof) and data

Functions of Operating Systems:

1. ***File Management:***

- File is the logical storage unit in a computer system.
- Files are usually organized into directories
- The operating system is responsible for the following activities in connection with file management:
  - Creating and deleting files and directories
  - Primitives to manipulate files and directories
  - Mapping files onto secondary storage
  - Controlling access control to files

Functions of Operating Systems:

1. ***Device Management:***

- An OS manages device communication via their respective drivers.
- It performs the following activities for device management:
  - Keeps tracks of all devices connected to system. designates a program responsible for every device known as the Input/output controller.
  - Decides which process gets access to a certain device and for how long.
  - Allocates devices in an effective and efficient way.
  - De-allocates devices when they are no longer required.

Functions of Operating Systems:

1. ***Network Management:***

- An OS manages active and passive components over network.
- Identifies faults in network connections and troubleshoots them.
- Enables accessing of remote resources.
- Provides proper security against viruses, hackers, and data corruption.
- Providing routing services, including support for major networking protocols, so that the operating system knows what data to send where.

Functions of Operating Systems:

1. ***Protection and Security***

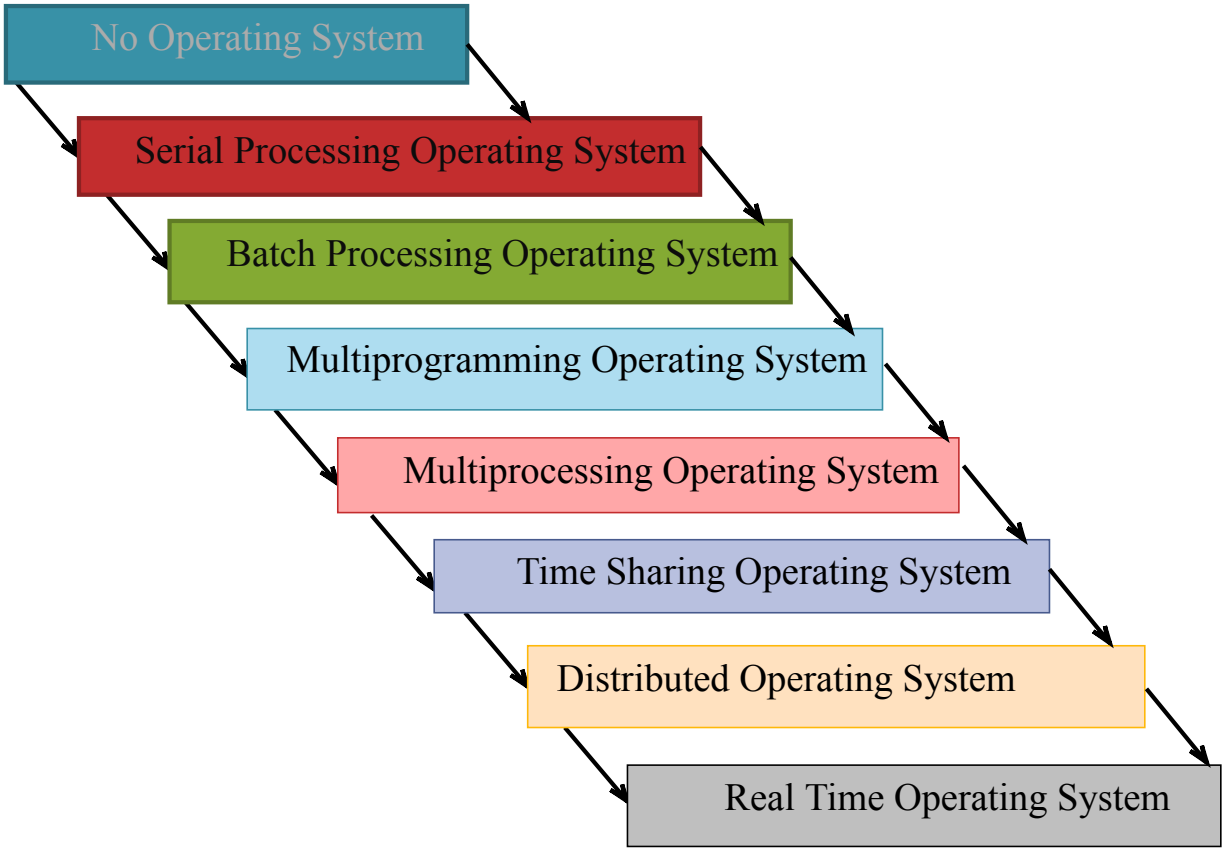
- An OS protects the computer system from malicious applications.
- OS regulates the flow of data within the system.
- It protects accessing of computer resources from unauthorized user.
- It uses password protection to protect user data and similar other techniques.

Functions of Operating Systems:

1. ***Provides User Interface***

- An OS facilitates the communication between an application and its user by acting as an intermediary between them.
- User interact with the system either by using Command-Line User Interface or by Graphical User Interface.
- **Command Line User** Interface displays text, and its commands are usually typed on a command line using a keyboard.
- With a **graphical user interface**, the functions are carried out by clicking or moving buttons, icons and menus by means of a pointing device.

Evolution of Operating Systems:



Era with no Operating System:

- In earlier machines, there were no any operating systems.
- In machines like PDP-11, a program coded in machine language are entered into RAM through front switches

Era with Serial Processing Operating System:

- With the advent of I/O devices like punch cards, magnetic tapes and with the advent of assembler and compilers, concept of serial processing operating system emerged.
- In serial processing, the programs were executed strictly in serial manner one after another.
- Program source code were written into assembly or high level language on a pack of punch cards and are entered into computer using card reader.
- The assembler or compiler are used to translate source code into object code.
- Object code links with library routines to create executable file which is then executed.

Era with Serial Processing Operating System:

- It is only after this that the next program was taken up for execution in a strictly serial manner.
- Advantages:
  - Resulted in a big improvement

Era with Batch Processing Operating System:

- Jobs with similar needs were batched together and were run through the computer as a group.
- These were the first “Mainframe” systems.
- Automatic job sequencing, automatic transfer from one job to another.
  - Job Control Language
- To over come the CPU idle time during transition from one job to next, Resident Monitor is created.
- Resident Monitor is a program that is held permanently in memory and controls sequence of events.

Era with Batch Processing Operating System:

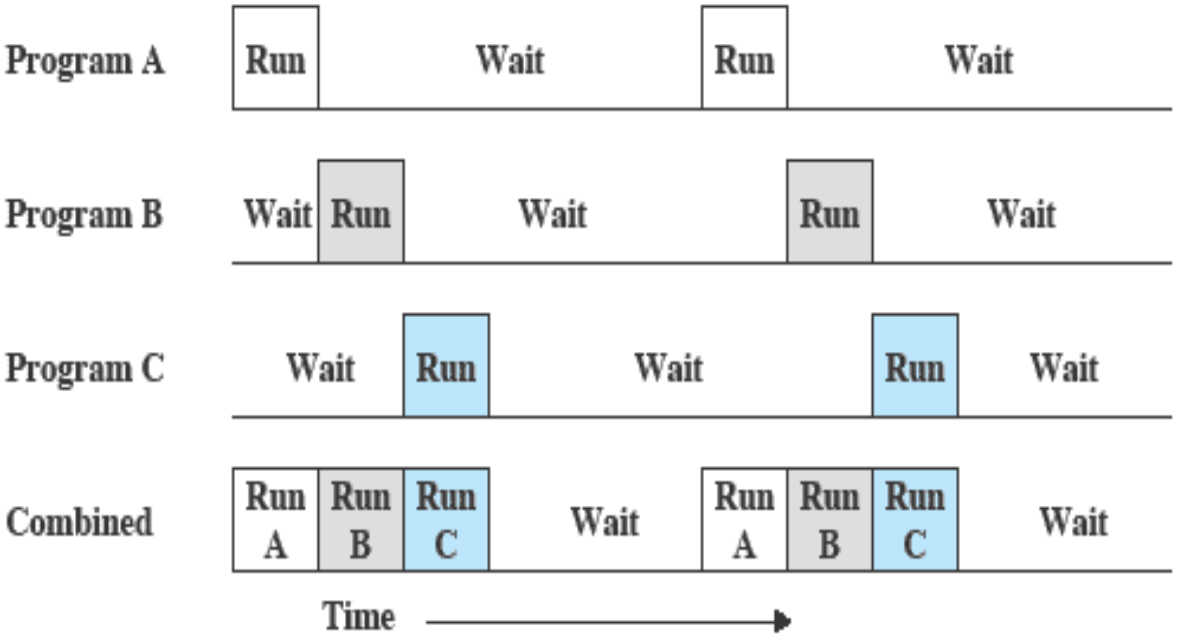
- Problem: Speed discrepancy between Fast CPU and Slow I/O devices emerged as a major bottleneck.
- Thus, there is a need to increase throughput and resource utilization by overlapping I/O and processing operations.
- Major development: ***DMA***
- However, CPU requires to be interrupted per block.
- Two approaches were introduced for improving system performances by overlapping I/O and CPU:

a. Buffering
b. Spooling

Era with Multiprogramming OS:

- It is a technique to execute number of programs simultaneously by a single processor.
- With fast CPU and slow I/O devices there is always a problem that during I/O operations, processor is idle.
- Solution:
  - Load more than one job into memory
  - Switch between jobs, whenever one of the jobs performs I/O operations
- Goal of multiprogramming
  - Maximize processor utilization

Multiprogramming



(c) Multiprogramming with three programs

Era with Multiprocessing OS:

- It is a technique to execute two or more process of same or different program by two or more processor in parallel.
- This type of OS handles multiple processors within a single Computer system which shares their bus, clock, memory and input/output devices.
- Facilitates the more effective utilization resources.
- If one of the CPUs breaks down, the other CPUs automatically controls the complete workload until repairs are made.
- Uses bigger size memory.

Era with Time Sharing OS:

- Also called Multitasking Operating System.
- A time sharing operating system allows multiple users to share computers simultaneously.
- Multiple users simultaneously access the system through terminals.
- They interact with a terminal session / shell that understands commands, allows to start programs.
- This kind of OS interleaves the execution of multiple user programs in round robin fashion.

Era with Time Sharing OS:

- Each user program is allocated a short burst or "quantum" of computation.
  - With n users online, each user will see 1/n effective computer capacity (there is operating system overhead)
  - As human reacting is slow compared to processor speed, such a shared computer's response time may be very similar to that of a dedicated computer
- Goals
  - Maximize CPU response time.

Era with Distributed OS:

- Distributed Operating system involves multiple computers, nodes, and sites, and these components are linked each other with a high speed LAN/WAN lines.
- Distributed OS is capable for sharing their computational capacity and I/O files with allowing the virtual machine abstraction to users.
- In this, a job is served to multiple computational node located at several geographical locations.

Era with Real Time OS:

- Real Time Operating system serves real time application that process data as it comes in, mostly without buffer delay.
- An RTOS is event-driven with no time wastage on processing time for the event which is not occur.
- In this type of system, processing must be done inside the specified constraints. Otherwise, the system will fail.
- It offers priority-based scheduling, which allows you to separate analytical processing from non-critical processing.

Reasons Behind Evolution of Several OS:

A major OS evolve over time for a number of reasons:

- Hardware upgrades
- New types of hardware
- New services
- Fixes

Types of Operating Systems:

According to the ***Technological Advancement:***

- Serial Processing OS
- Batch Processing OS
- Multi-Programming OS
- Multi-Tasking OS
- Multi-Processing OS
- Multi-Threading OS
- Time Sharing OS
- Real Time OS
- Network OS
- Distributed OS

Types of Operating Systems:

According to the ***User Interface:***

- CUI Based OS
- GUI Based OS

Operating System Structure:

Some of the main structures used in operating systems are:

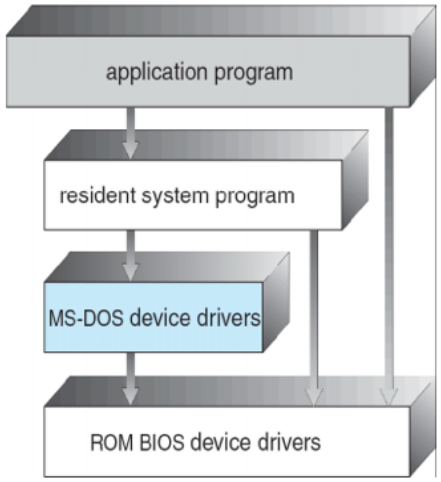
- Monolithic Structure
- Layered Structure
- Micro Kernel Structure
- Virtual Machine Structure
- Client/Server Structure
- Exo-Kernal Structure

Monolithic Structure of OS:

- It is oldest architecture used for developing OS.
- Used to perform small tasks.
- In this, OS resides on kernel for anyone to execute.
- Components needed for processing are embedded into the kernel and are controlled by the kernel.
- Examples:
- MS-DOS Structure
- Unix System Structure
- Linux System Structure
- OS/360

MS-DOS Structure:

- MS-DOS was written to provide the most functionality in the least space.
- Not divided into modules.
- Although MS-DOS has some structure, its interfaces and levels of functionality are not well separated.
- For instance, application programs are able to access basic I/O routines to write directly on display and disk drives.



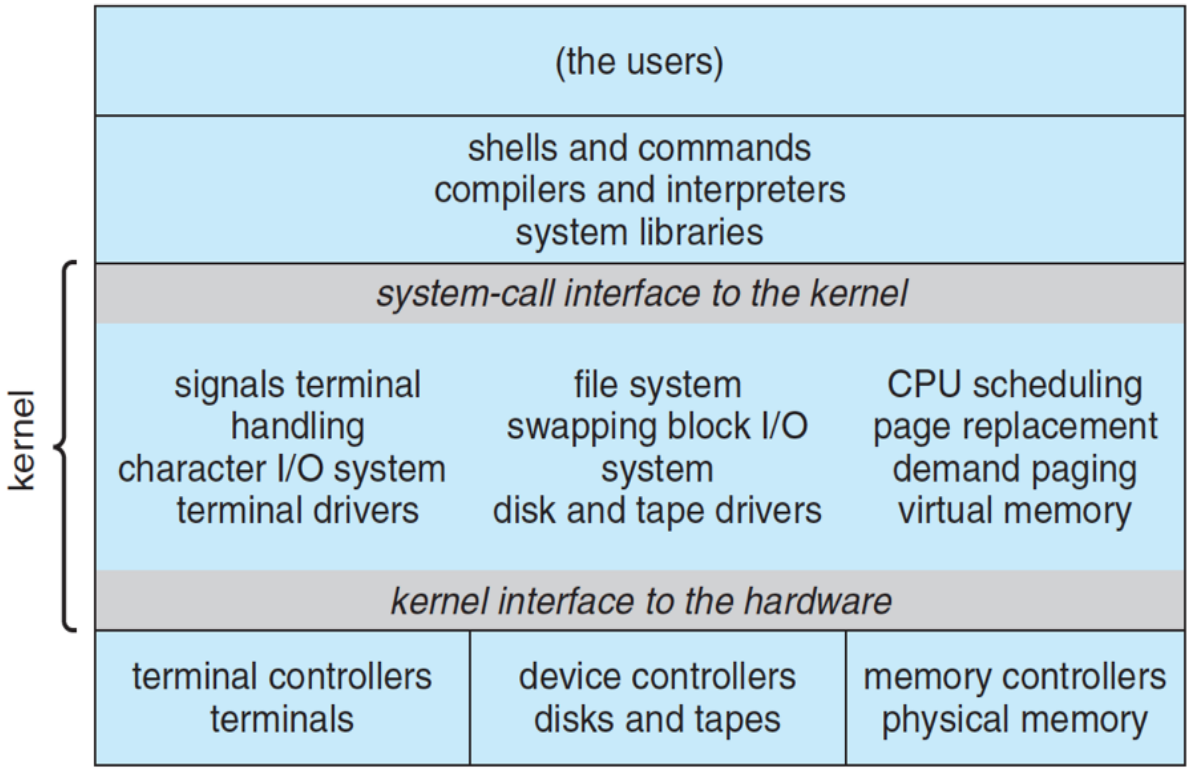
MS-DOS layer structure

- This leaves vulnerable to errant programs, causing entire system crashes when user programs fail.

Unix System Structure:

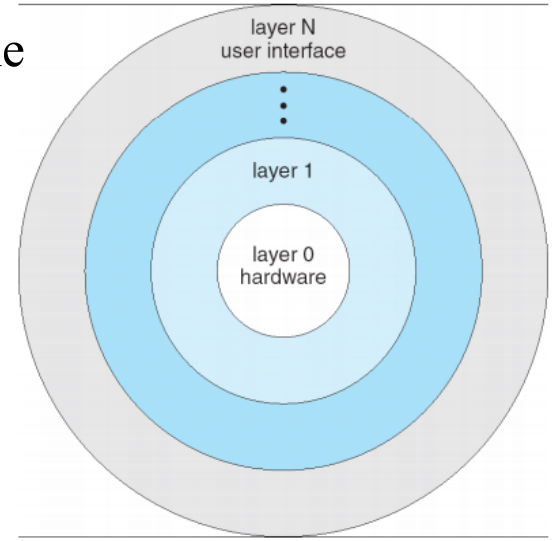
- UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring. The UNIX OS consists of two separable parts:
  - **Systems programs**
    - Systems programs are provided to define the user interface.
  - **The kernel**
    - Consists of everything below the system-call interface and above the physical hardware.
    - Separated into a series of interfaces and device drivers.
    - Different components (file system, CPU scheduling, memory management and other OS functions) were layered in its design.
  - OS may provide smaller modules for much greater control of computer resources, incorporating changes in the separate modules easily to enhance functionality and hide information in different components.

Unix System Structure:



Layered Structure of OS:

- OS is divided into a number of layers (levels), each built on top of lower layers.
- The bottom layer (layer 0) is the hardware; the highest (layer N) is the user interface.



- With modularity, layers are selected such that each uses functions and services of only lower-level layers.
- OS has a greater control over the computer resources and applications that make use of it.

Layered Structure of OS:

- Provides more freedom to the implementers to make changes in the inner workings of the system.
- Provides information hiding by allowing the programmers to implement the low-level routines using top-down modular programming approach.
- Layered approach is used for modularization of the OS.
- Higher-level layers can invoke lower-level layers.
- Less number of layers provide more functionality at each level while reducing the problems.

Layered Structure of OS:

- ***Advantages:*** modularization makes debugging and verification much easier; design and implementation is simplified; each layer hides certain data structures, operations and hardware details from higher-level layers.
- ***Disadvantages:*** level definition and implementation is very difficult; order of levels for the design of layered structure is problematical; interaction between layers and passing of parameters / data is very complex; less efficient as a system call takes longer time for execution than a non-layered system.
- ***Examples:*** Technische Hogeschool Eindhoven (THE) OS and Venus layer structure.

Layered Structure of OS:

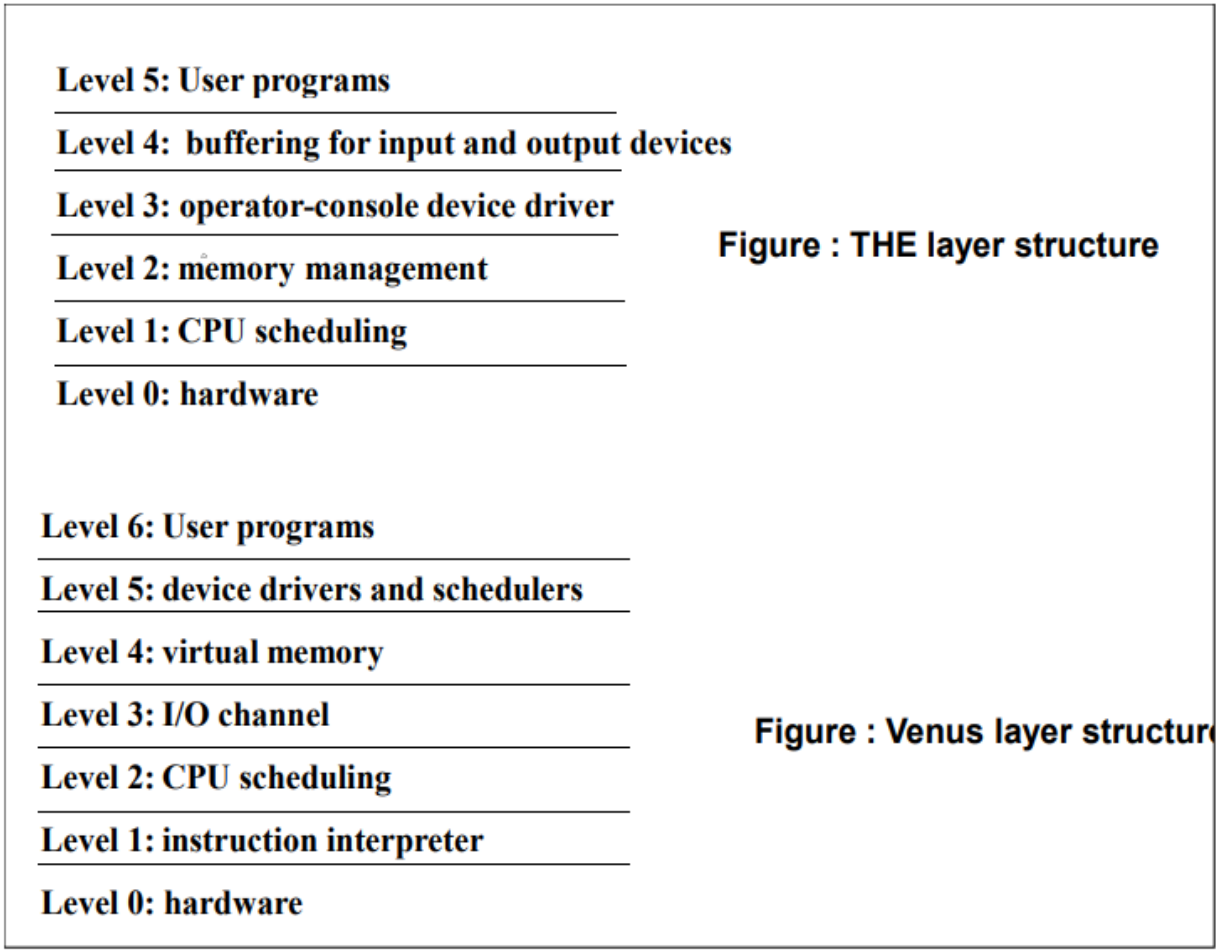


Figure : THE layer structure

Figure : Venus layer structur

Micro Kernel Structure of OS:

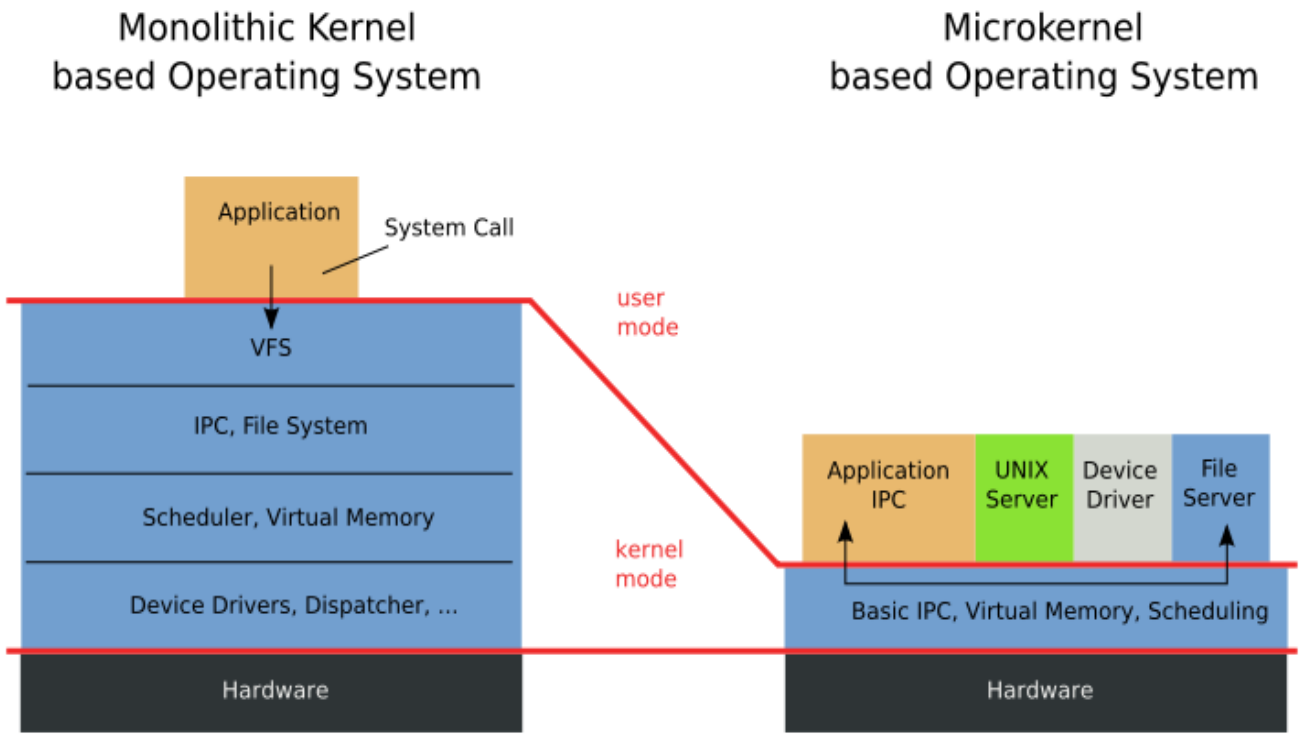
- Larger kernels need more space and are difficult to manage.
- In the mid-1980s, researchers at Carnegie Mellon University developed an operating system called Mach that modularized the kernel using the microkernel approach.
- This method structures the operating system by removing all nonessential components from the kernel and implementing them as system and user-level programs.
- The result is a smaller kernel.
- Microkernel provides provide minimal process and memory management, in addition to a communication facility.

Micro Kernel Structure of OS:

- The main function of the microkernel is to provide a communication facility between the client program and the various services that are also running in user space.

- Benefits:
  - easier to extend a microkernel
  - fewer changes required in the modification of microkernel
  - easier to port the operating system to new architectures
  - more reliable (less code is running in kernel mode)
  - more secure – if a routine fails, the rest of the OS remains untouched

Micro Kernel Structure of OS:

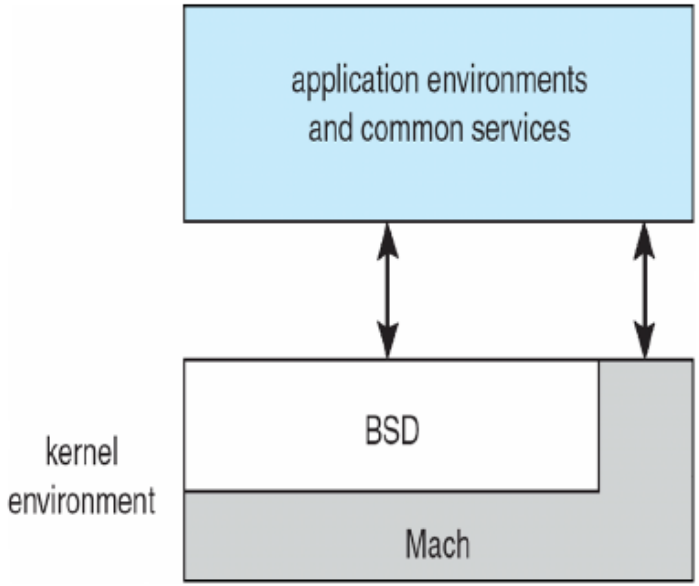


Micro Kernel Structure of OS:

- Example: Mac OS X
- Mac OS X (also known as Darwin) structures the operating system using a layered technique .
- The top layers include application environments and a set of services providing a graphical interface to applications.
- Below these layers is the kernel environment, which consists primarily of the Mach microkernel and the BSD kernel.
- Mach provides memory management; support for remote procedure calls (RPC) and inter process communication (IPC) facilities, including message passing; thread scheduling.

Micro Kernel Structure of OS:

- The BSD component provides a BSD command line interface, support for networking and file systems, and an implementation of POSIX APIs, including threads.



Virtual Machine Structure:

- The layered approach is taken to its logical conclusion in the concept of a virtual machine.
- The virtual machine concept treats the kernel of the OS and the hardware though they were all hardware.
- System programs treat hardware instructions and system calls as they were at the same level. Application programs can also call system programs.
- A virtual machine provides an interface identical to the underlying bare hardware.
- The operating system creates the illusion of multiple processes, each executing on its own processor with its own (virtual) memory.

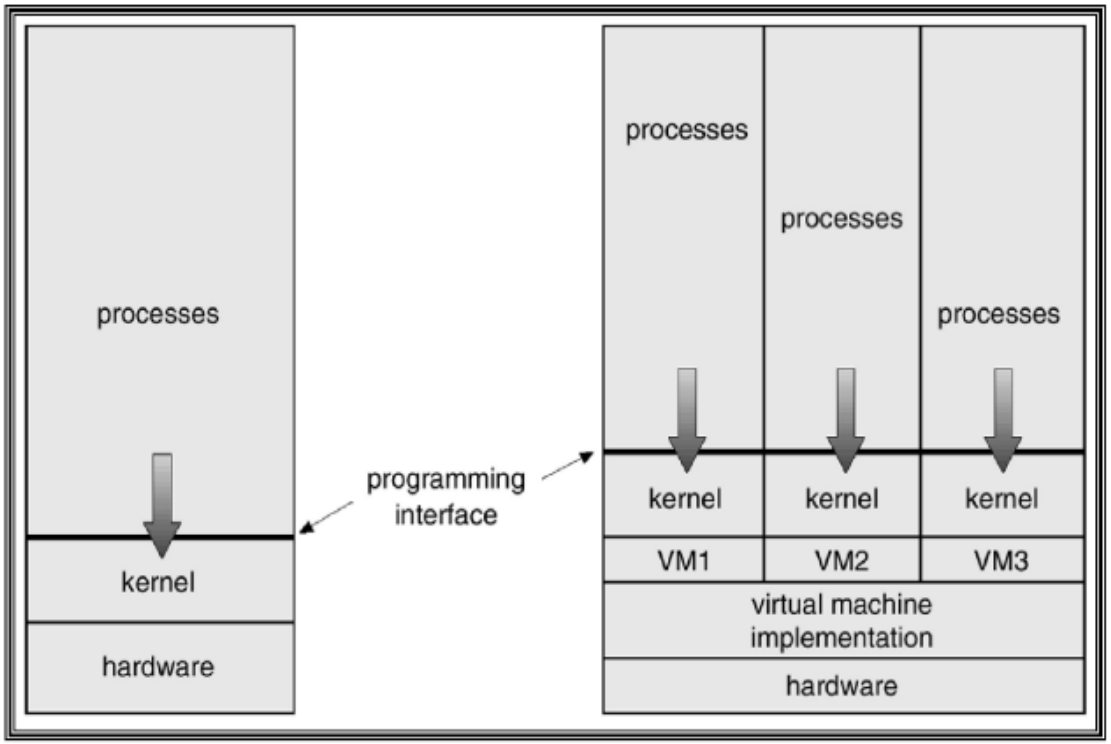
Virtual Machine Structure:

- The resources of the physical computer are shared to create the virtual machines:
  - CPU scheduling can create the appearance that users have their own processor.
  - Spooling and a file system can provide virtual card readers and virtual line printers.
  - A normal user time-sharing terminal serves as the virtual machine operator's console.
- A virtual machine has virtual memory, virtual disks, virtual printer, virtual console, virtual processor and virtual communication network.
- A user can run any type of software package on his/her virtual machine.

Virtual Machine Structure:

- The virtual machine software is concerned with multiprogramming multiple virtual machines onto a physical machine.
- Virtual disks (minidisks) are difficult to implement in a virtual machine.
- A lot of effort is required to provide an exact duplicate of the underlying machine.
- Virtual user mode and virtual monitor mode run in the physical user mode.

Virtual Machine Structure:



Virtual Machine Structure:

- **Advantages**
- The virtual-machine concept provides complete protection of system resources since each virtual machine is isolated from all other virtual machines.
- A virtual-machine system is a perfect vehicle for operating systems research and development. System development is done on the virtual machine, instead of on a physical machine and so does not disrupt normal system operation.
- Virtual machines are useful for solving system compatibility problems.

- **Disadvantages**
- No direct sharing of resources as each virtual machine is isolated from all other virtual machines.
- The virtual machine concept is difficult to implement due to the effort required to provide an exact duplicate to the underlying machine.
- The privileged instructions (needed mainly for I/O) must be simulated and hence execute more slowly.

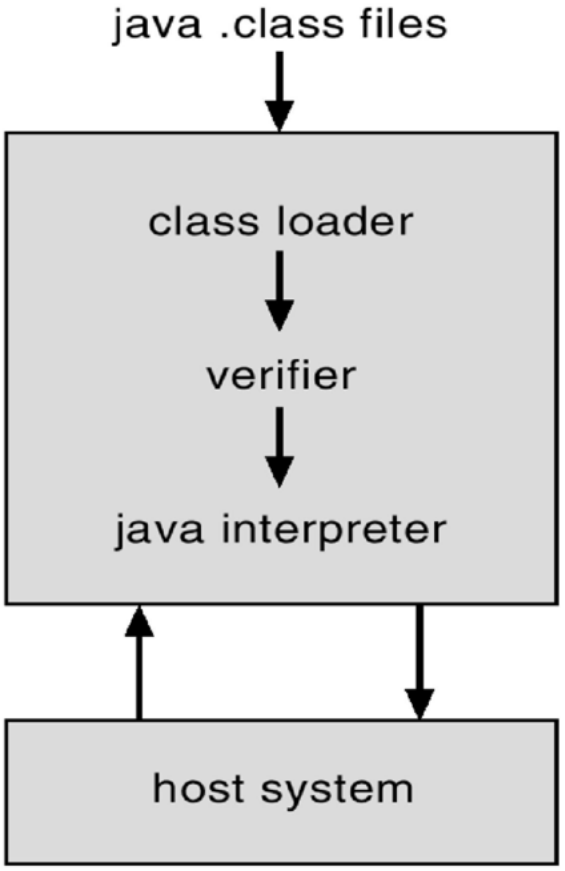
Virtual Machine Structure:

- Example: ***Java Virtual Machine***
- In addition to a language specification and a large API library, Java also provides a specification for a Java Virtual Machine (JVM).
- For each Java class, the Java compiler produces an architecture neutral byte code output (.class) file that will run on any implementation of the JVM.
- JVM consists of:
  - class loader
  - class verifier
  - runtime interpreter

Virtual Machine Structure:

- The Java interpreter may be a software module that interprets the byte-codes one at a time, or it may be a Just-In-Time (JIT) compiler that turns the architecture-neutral byte codes into native machine language for the host computer.
- Most implementations of the JVM use a JIT compiler for enhanced performance.
- JVM design provides a secure, efficient, object-oriented, portable, and architecture-neutral platform on which to run Java programs.

Java Virtual Machine

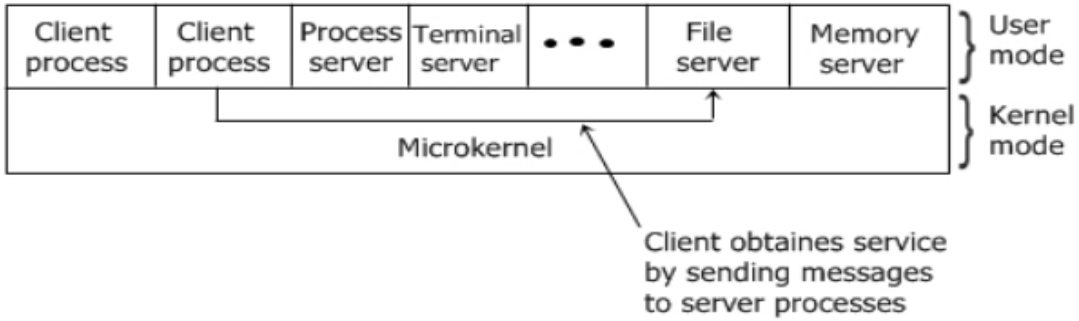


Client/Server Structure:

- Client/Server architecture is a trend in modern operating system.
- Client/server model works on request- response technique.
- In this, client always request services for user process to server and on the other side, server gates complementing that request and send response back to the client.
- For example; client sends request to read a particular file to the server process, server checks the parameter and finds whether it is valid or not, after that server does the work and send back the answer to client.
- The figure below shows client server architecture.

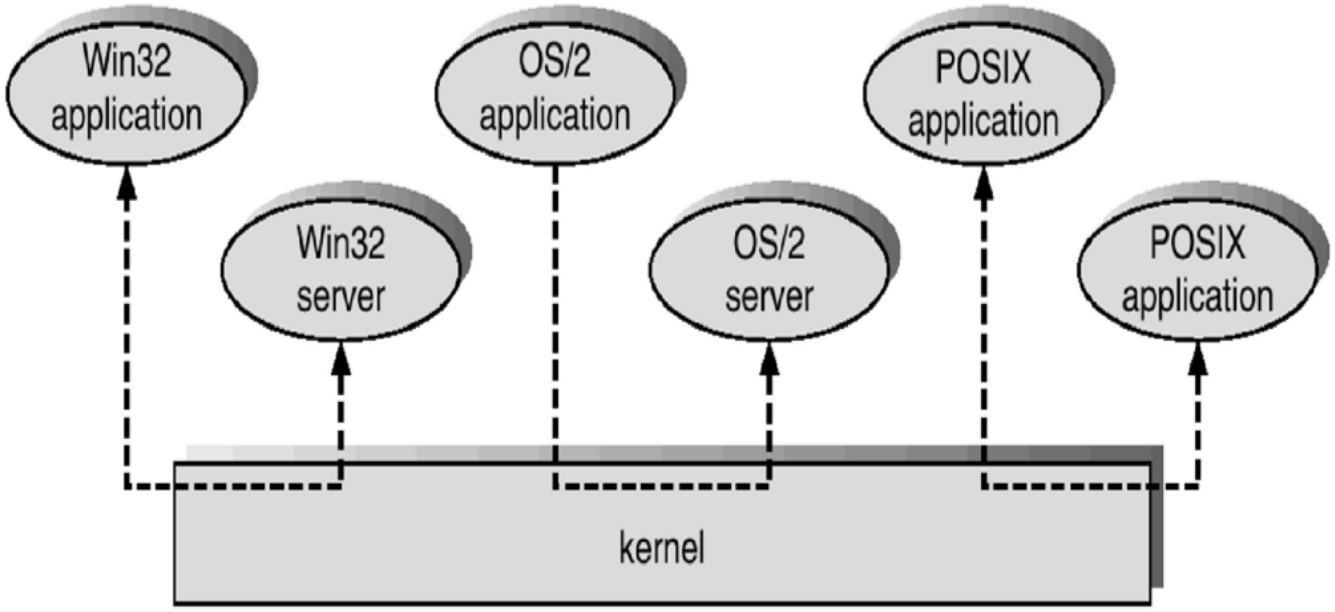
Client/Server Structure:

- In this model, the main task of the kernel is to handle all the communication between the client and the server by splitting the operating system into number of ports, each of which only handle some specific task. I.e. file server, process server, terminal server and memory service.
- The figure below shows client server architecture.



- Advantage of the client-server model is it's adaptability to user in distributed system.

Windows NT Client-Server Structure



Exo-Kernel Structure:

- Developed at MIT to provide application-level management of hardware resources.
- By separating resource management from protection, the exokernel architecture aims to enable application-specific customization.
- Due to its limited operability, exokernel size typically tends to be minimal.
- The OS will always have an impact on the functionality, performance and scope of the apps that are developed on it because it sits in between the software and the hardware.
- The exokernel operating system makes an attempt to address this problem by rejecting the notion that an operating system must provide abstractions upon which to base applications.
- The objective is to limit developers use of abstractions as little as possible while still giving them freedom.

Exo-Kernel Structure:

Advantages:

- **High Performance** - As application program can allocate memory, a better designed code can make optimal use and perform better.
- **Application Control** - As resource management is not secured by operating system, application program has more control over system resources and can write custom operations on system resources.

Disadvantages:

- **Unreliable and Unsafe** - As security is in application program level, a poorly written code can ruin the system.
- **Complex Design** - Exo-Kernel designing is complicated.

System Calls

- System calls provide the interface between a process and the OS:
- Generally available as assembly-language instructions.
- Languages defined to replace assembly language for systems programming allow system calls to be made directly (e.g., C, C++).
- System calls are mostly accessed by programs via a high level Application Program Interface (API) rather than direct system call use
- API specifies a set of functions that are available to an application programmer, including the parameters that are passed
- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM)

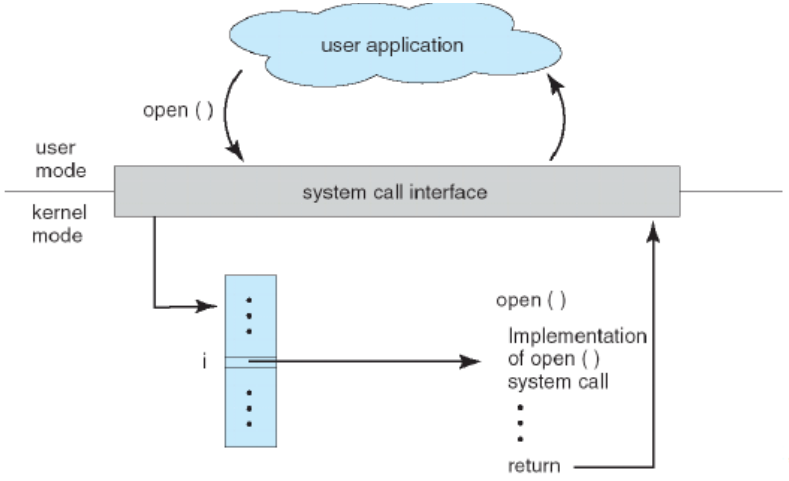
System Calls

- A system call is used to complete a specific task:
  - Reading input file name
  - Opening a file
  - Error message if any
  - Abnormal termination
  - Deletion of a file
  - Prompting a message
  - Reading data for the file
  - Writing data to the file or console
  - Closing the file

- Why use APIs rather than system calls?
  - Program portability
  - Actual system calls can be more detailed and difficult to work with than the API

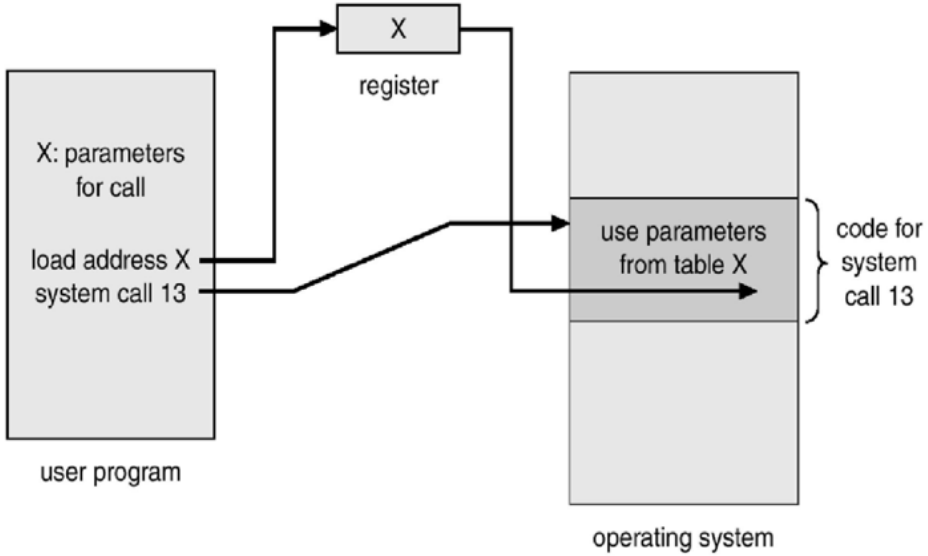
System Call Implementation

- Programming languages provides a system-call interface that serves as the link to system calls made available by the OS
- System-call interface intercepts function calls in the API and invokes the necessary system call within the OS
- System-call interface maintains a table indexed according to these numbers associated with each system call
- The system call interface invokes intended system call in OS kernel and returns status of the system call and any return values



System Call Parameter Passing

- Three general methods are used to pass parameters between a running program and the operating system:
  - Pass parameters in registers.
  - Store the parameters in a table in memory, and the table address is passed as a parameter in a register (used in Linux)
  - Push (store) the parameters onto the stack by the program, and pop off the stack by OS.



Types of System Calls

1. **Process control**
   - End, Abort
   - Load, Execute
   - Create process, Terminate process
   - Get process attributes, Set process attributes
   - Wait for time
   - Wait event, Signal event
   - Allocate memory, Free memory
2. **File management**
   - Create file, Delete file
   - Open, Close
   - Read, Write, Reposition
   - Get file attributes, Set file attributes

Types of System Calls

1. **Device management**

- Request device, Release device
- Read, Write, Reposition.
- Get device attributes, Set device attributes.
- Logically attach devices, Logically detach devices.

2. **Information maintenance**

- Get time or date, Set time or date
- Get system data, Set system data
- Get process, file, or device attributes, Set process, file, or device attributes.

Types of System Calls

1. **Communications**

- Create communication connection, Delete communication connection
- Send messages, Receive messages
- Transfer status information
- Attach remote devices, Detach remote devices

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

	Windows	Unix
Process Control	CreateProcess()	fork()
	ExitProcess()	exit()
	WaitForSingleObject()	wait()
File Manipulation	CreateFile()	open()
	ReadFile()	read()
	WriteFile()	write()
	CloseHandle()	close()
Device Manipulation	SetConsoleMode()	ioctl()
	ReadConsole()	read()
	WriteConsole()	write()
Information Maintenance	GetCurrentProcessID()	getpid()
	SetTimer()	alarm()
	Sleep()	sleep()
Communication	CreatePipe()	pipe()
	CreateFileMapping()	shmget()
	MapViewOfFile()	mmap()
Protection	SetFileSecurity()	chmod()
	InitlializeSecurityDescriptor()	umask()
	SetSecurityDescriptorGroup()	chown()